# From Features to Machine Learning

Week 12 — Session 1: Image features & the recognition pipeline

Practical Image Processing

Lecture 12.1 ▪ 60–70 min

# A turning point in the course

- Weeks 1–11: *we* designed every step (filters, edges, corners, warps).
- Now: we let the computer *learn* the decision from labelled examples.
- The bridge: turn an image into a **feature vector**, then train a **classifier**.

**Today's real task**

Classify Iranian car models (206 / 405 / Dena) from **HOG** features — using your `E:/data` images.

# Where Session 1 stops

- **Session 1 (today):** how to turn an image into a *good* feature vector, and what a classifier is.
- **Session 2 (next):** training, honest evaluation, overfitting, and the limits of hand-crafted features.

**Key idea**

You cannot fix a bad classifier with more training. Everything starts with the features. Today is about features.

# Roadmap

The recognition pipeline

Gradients: the raw material of HOG

HOG step by step

Beyond HOG

What a classifier is

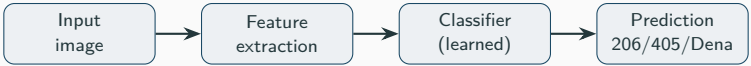
Train / test discipline

In-class exercise

Wrap-up

# The recognition pipeline

# The classic recognition pipeline




1. **Input image** $\rightarrow$
2. **Feature extraction** (turn pixels into a compact vector) $\rightarrow$
3. **Classifier** (a learned function: vector $\rightarrow$ label) $\rightarrow$
4. **Prediction**.

**Key idea**

Until $\sim$2012, essentially all recognition worked this way. The features were hand-designed; only the classifier was learned.

# Why not feed raw pixels?

- A $96 \times 96$ image is 9216 numbers — huge, and every shift/lighting change moves them all.
- Raw pixels are not **invariant**: the same car, moved 5 px, looks "different".
- **Features** (HOG, colour histograms, ORB) compress the image into a stable, meaningful vector.

**Math**

Shift a $96 \times 96$ image by one pixel: almost all 9216 values change, yet it is the *same* car. A raw-pixel distance would call it far away. Good features must not.

# The curse of dimensionality

- In 9216 dimensions, "nearest neighbour" becomes almost meaningless — distances all look similar.
- You need exponentially more examples to fill a high-dimensional space.
- Features cut 9216 $\rightarrow$ a few hundred meaningful numbers.

**✗ More dimensions than samples**

126 training images but a 9216-D pixel vector? The model has enough freedom to memorize noise.

# What makes a good feature

- **Invariant** to nuisances: small shifts, lighting, minor rotation.
- **Discriminative**: different classes land in different regions.
- **Compact**: a few hundred numbers, not ten thousand.

**Math**

Good features make classes **separable**: similar things land near each other, different things land apart — in a space small enough to learn a boundary.

# Gradients: the raw material of HOG

# Gradients recap — where edges point

- Recall the gradient $\nabla I = (I_x, I_y)$ from the edges week.
- **Magnitude** $= \sqrt{I_x^2 + I_y^2}$ says *how strong* the edge is.
- **Orientation** $= \arctan(I_y / I_x)$ says *which way* it points.

HOG throws away the exact pixels and keeps only *orientation*, weighted by magnitude.




gradient vectors per pixel

# From gradient to orientation — a worked pixel

A pixel with horizontal and vertical differences:

$$I_x = 30, \qquad I_y = 40.$$

**Math**

Magnitude $= \sqrt{30^2 + 40^2} = \sqrt{2500} = 50$.

Orientation $= \arctan\left(\frac{40}{30}\right) \approx 53^\circ$.

- This pixel votes for the "53°" bin with weight 50.
- Using 9 bins over 0–180° (unsigned), each bin spans 20°.
- 53° falls in the 40–60° bin (and a little into 60–80° by interpolation).

**Key idea**

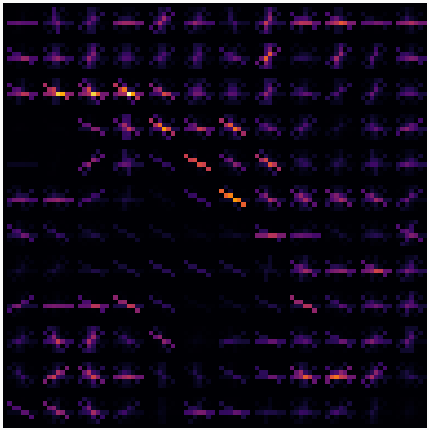
Every pixel casts a magnitude-weighted vote into the orientation bin it points to.

# HOG step by step

# HOG: Histogram of Oriented Gradients



a real 206



its HOG (edge-orientation) map

HOG divides the image into **cells** and, in each, histograms the gradient directions — a compact shape signature.

# The HOG algorithm, step by step

1. Compute gradients $I_x, I_y$ at every pixel.
2. Get magnitude and orientation per pixel.
3. Split the image into small **cells** (e.g. $8 \times 8$ px).
4. In each cell, build a 9-bin **histogram** of orientations, weighted by magnitude.
5. Group cells into **blocks** and **normalize** (for lighting invariance).
6. Concatenate all block histograms $\rightarrow$ one long feature vector.

**Key idea**

Steps 1–2 are last week's gradients. Steps 3–6 are what makes it a robust, fixed-length descriptor.

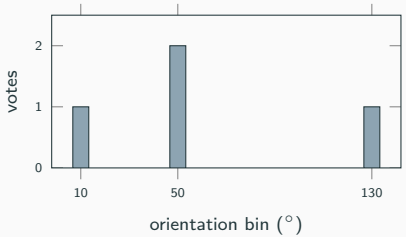
# A worked HOG cell

One $2 \times 2$ patch of orientations (degrees), all magnitude $= 1$ for simplicity:

| | |
|---|---|
| 10 | 50 |
| 50 | 130 |

Bin width $20^\circ$, bins centred at $10, 30, \ldots, 170$.

Votes into the 9-bin histogram:




Two pixels point at $50^\circ$, so that bin gets 2. The result is a 9-number signature for this cell.

# Block normalization — why it matters

- Raw histogram values scale with *contrast*: a sunny photo has bigger gradients everywhere.
- Group nearby cells into a **block** and divide by the block's norm.
- Now the descriptor depends on edge *directions*, not on absolute brightness.

**Math**

$L2$ normalization: $v \leftarrow \dfrac{v}{\sqrt{\|v\|^2 + \epsilon}}$.
The tiny $\epsilon$ avoids dividing by zero in flat regions.

**Good practice**

Overlapping blocks (each cell counted in several blocks) is what makes real HOG robust to lighting.

# Why HOG works for shape

- Cars differ in *shape* (roofline, lights, grille) — captured by gradient directions.
- Histogramming per cell gives tolerance to small shifts and lighting changes.
- HOG + linear SVM was the state of the art for pedestrian/vehicle detection for years.

**Key idea**

HOG throws away exact pixel values and keeps *where edges point* — the essence of shape.

# HOG hyper-parameters

| Knob | Typical | Effect |
|---|---|---|
| Cell size | $8 \times 8$ px | smaller = finer detail, longer vector |
| Orientation bins | 9 | more bins = finer angles, more noise |
| Block size | $2 \times 2$ cells | bigger = stronger normalization |
| Signed angles? | no (0–180°) | unsigned is usual for object shape |

**Vector length**

A $96 \times 96$ image, $8 \times 8$ cells, $2 \times 2$ blocks, 9 bins $\Rightarrow$ roughly 900 numbers — the "900-D" vectors we train on.

# Beyond HOG

# Colour histograms — a complementary feature

- HOG is grayscale — it ignores colour entirely.
- Some car models cluster around typical colours; a **colour histogram** captures that.
- Count pixels per colour bin (e.g. hue), normalize by pixel count.

Concatenate HOG + colour to get a richer vector.



# Other classical descriptors

| Descriptor | Captures | Good for |
|---|---|---|
| HOG | edge orientations | overall shape |
| LBP | local texture patterns | surface/texture, faces |
| Colour hist. | colour distribution | colour-distinct classes |
| ORB / SIFT | keypoints & matches | alignment, stitching |

**Key idea**

There is no single best feature. You pick (or combine) descriptors that expose what separates *your* classes.

# What a classifier is

# What a classifier learns

- Given feature vectors with labels, it learns a **decision boundary** between classes.
- **k-NN**: label a point by the majority vote of its $k$ nearest training points.
- **SVM**: find the boundary with the widest **margin** between classes.

**Math**

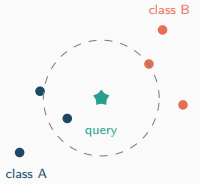
k-NN *memorizes*; SVM *optimizes* a margin. Both need features where classes are separable.

# k-NN in detail

- "Training" just stores all labelled feature vectors.
- To predict: find the $k$ closest stored points, take a majority vote.
- Distance is usually Euclidean in feature space.

**✗ Scale sensitivity**

If one feature has a huge range, it dominates the distance. Standardize first.




$k$=3: nearest neighbours vote

# A worked k-NN vote

Query at feature vector $q$. Distances to 5 nearest training points:

| neighbour | distance | label |
|---|---|---|
| 1 | 0.8 | 206 |
| 2 | 1.1 | 206 |
| 3 | 1.3 | Dena |
| 4 | 1.4 | 206 |
| 5 | 1.6 | 405 |

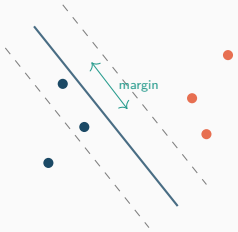
- With $k = 5$: votes are 206$\times$3, Dena$\times$1, 405$\times$1.
- Majority $\Rightarrow$ predict **206**.
- With $k = 1$ it would still be 206 (closest is 206).

**Key idea**

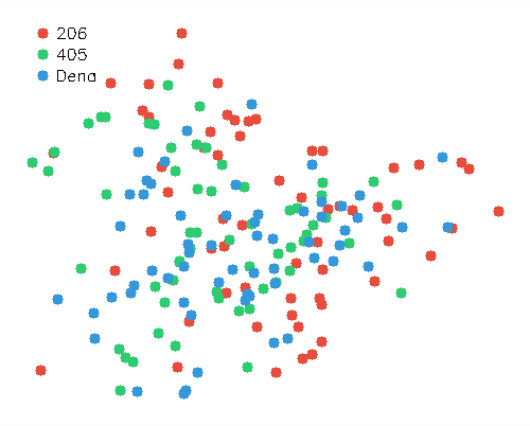
Small $k$ = flexible but noisy; large $k$ = smooth but can wash out small classes.

# SVM & the margin

- An SVM finds the separating line with the **widest gap** to the nearest points.
- Those nearest points are the **support vectors**.
- A wide margin generalizes better than a boundary squeezed against the data.
- The **RBF kernel** lets the boundary curve for non-linear data.



# Our classes in feature space (PCA to 2D)




- Each dot is a real car's HOG vector, projected to 2D with PCA.
- Classes overlap a lot — these three models genuinely look similar. Hard problem!

# Feature scaling — a quiet but critical step

**✔ Standardize**

Rescale each feature to zero mean, unit variance *before* k-NN or an RBF SVM.

**✘ Leakage**

Fit the scaler on *training only*, then apply to test. Fitting on all data leaks test info.

**Key idea**

Distance-based methods are only as good as their scaling. HOG bins are already comparable, but mixing HOG with colour or size makes scaling essential.

# Train / test discipline

# Train / test split — the golden rule

- Split data: **train** the classifier on one part, **test** on unseen data.
- Testing on training data lies — the model can just memorize.
- We used 70% train / 30% test, **stratified** (each class balanced in both).

**✕ Pitfall**

Never report accuracy on data the model has seen. That number is meaningless for real use.

# Stratification & data leakage

- **Stratified** split keeps each class's proportion in both train and test.
- Without it, a small class might vanish from the test set.
- **Leakage**: any test information sneaking into training inflates your score.

**Common leaks**

Scaling on all data, near-duplicate frames of the same car in both splits, tuning on the test set.

# In-class exercise

# In-class exercise (10 min)

**Design the features**

1. A pixel has $I_x = 30$, $I_y = 40$. Give its gradient magnitude and (roughly) its orientation.
2. You must separate white 206s from black 405s. Which feature helps most — HOG or colour? Why?
3. Your HOG vector is 900-D but you have 126 training images. Name one risk and one fix.

# Debrief

- (1) Magnitude $= \sqrt{30^2 + 40^2} = 50$; orientation $= \arctan(40/30) \approx 53°$.
- (2) **Colour histograms** — the models differ mainly by colour here, and HOG is grayscale. Best: combine both.
- (3) Far more dimensions than samples $\Rightarrow$ overfitting risk; fix with PCA / fewer HOG cells / more data.

**Key idea**

Choosing the right feature is half the battle — before any classifier is trained.

# Wrap-up

# Session 1 — what you can now do

- Explain why raw pixels are a poor representation, and what makes a feature good.
- Compute a gradient's magnitude and orientation, and describe the HOG pipeline end to end.
- Combine HOG with colour or texture descriptors when needed.
- Describe k-NN and SVM, and set up a correct stratified train/test split.

# Next session

**Session 2 — training & the truth about scores**

- Actually train k-NN and SVM in code on `E:/data`.
- Read our real results — and discover a 100% train / 48% test gap.
- Overfitting, confusion matrices, precision/recall, and cross-validation.
- Why hand-crafted features hit a ceiling — the doorway to deep learning.

No deliverable yet — bring your feature-extraction code from today.

## Good features come first.

**A classifier can only draw a boundary in the space your features give it — next session we train one and see how honest it really is.**